

Advanced Cypher Queries for Social Botnet Analytics

1. Community Sizes

Counts the number of users in each community (cluster).

```
MATCH (n:User)
RETURN n.cluster AS community, count(*) AS size
ORDER BY size DESC
```

2. Bots per Community

Shows how many bots exist in each community.

```
MATCH (n:User)
WHERE n.is_bot = true
RETURN n.cluster AS community, count(*) AS num_bots
ORDER BY num_bots DESC
```

3. Show All Bots and Their Communities

Lists all bot users and the community (cluster) they belong to.

```
MATCH (n:User)
WHERE n.is_bot = true
RETURN n.user_id AS user_id, n.cluster AS community
ORDER BY community
```

4. Most Connected (Influential) Nodes (Total Degree Centrality)

Finds the top 10 users with the most total connections (both in and out).

```
MATCH (n:User)
OPTIONAL MATCH (n)-[r]-()
RETURN n.user_id AS user_id, n.cluster AS community, count(r) AS total_degree
ORDER BY total_degree DESC
LIMIT 10
```

5. Communities With Both Bots and Humans

Shows which communities have both bots and humans.

```
MATCH (n:User)
WITH n.cluster AS community, collect(DISTINCT n.is_bot) AS types
WHERE size(types) > 1
RETURN community, types
ORDER BY community
```

6. Inter-Community Bridges (Edges Linking Different Communities)

Lists edges that connect users from different communities.

```
MATCH (a:User)-[r:FOLLOWS]-(b:User)
WHERE a.cluster <> b.cluster
RETURN a.user_id AS from_user, a.cluster AS from_comm, b.user_id AS to_user, b.cluster AS to_comm
LIMIT 25
```

7. Visualize a Specific Community (e.g., Community 4)

Visualizes community 4 and its direct connections.

```
MATCH (n:User)-[r:FOLLOWS]-(m:User)
WHERE n.cluster = 4 OR m.cluster = 4
RETURN n, r, m
LIMIT 300
```

8. Subgraph of Only Bots

Shows how bots are interconnected.

```
MATCH (n:User)-[r:FOLLOWS]-(m:User)
WHERE n.is_bot = true AND m.is_bot = true
RETURN n, r, m
LIMIT 300
```

9. Bots vs Humans (Entire Network)

Simple count of bots vs humans in your dataset.

```
MATCH (n:User)
RETURN CASE WHEN n.is_bot = true THEN 'bot' ELSE 'human' END AS type, count(*) AS num_users
ORDER BY num_users DESC
```

10. Top 10 Communities by Bot Ratio (Only Where Size >= 20)

Highlights communities that are most bot-dense.

```
MATCH (u:User)
WITH u.cluster AS cluster,
     sum(CASE WHEN u.is_bot THEN 1 ELSE 0 END) AS botCount,
     count(*) AS size
WHERE size >= 20
RETURN cluster, size, toFloat(botCount)/size AS botRatio, botCount
ORDER BY botRatio DESC
LIMIT 10
```

11. Inspect a Cluster (e.g., Cluster 39) Out-Degree

Top 5 most outward-following users in cluster 39.

```
MATCH (u:User)-[r:FOLLOWS]->(v:User)
```

```
WHERE u.cluster = 39
RETURN u.user_id AS user, count(v) AS outDegree
ORDER BY outDegree DESC
LIMIT 5
```

12. Target Hubs Inside a Cluster (e.g., Cluster 39) In-Degree

Who is followed most by users from cluster 39.

```
MATCH (u:User)-[:FOLLOWS]->(v:User)
WHERE u.cluster = 39
RETURN v.user_id AS target, count(u) AS inDegreeFromBotCluster
ORDER BY inDegreeFromBotCluster DESC
LIMIT 5
```

13. Inspect Top 5 Target Accounts

Details about top 5 targeted accounts.

```
MATCH (u:User)
WHERE u.user_id IN [3632, 6678, 7700, 9300, 9843]
RETURN u.user_id AS user,
       CASE WHEN u.is_bot = true THEN 'bot' ELSE 'human' END AS type,
       u.stance AS stance,
       u.cluster AS cluster
ORDER BY user
```

14. Stance Breakdown for a Cluster (e.g., Cluster 39)

Distribution of stances within a community.

```
MATCH (u:User)
WHERE u.cluster = 39
RETURN u.stance AS stance, count(*) AS count
ORDER BY count DESC
```

15. Top 10 Communities Targeted by a Cluster (e.g., Cluster 39)

Top 10 communities that cluster 39 users follow the most.

```
MATCH (u:User)-[:FOLLOWS]->(v:User)
WHERE u.cluster = 39 AND v.cluster <> 39
RETURN v.cluster AS targetCluster, count(*) AS followCount
ORDER BY followCount DESC
LIMIT 10
```

16. Profile the Top Targeted Community (e.g., Cluster 86)

A. Size and Bot Rate

```

MATCH (u:User)
WHERE u.cluster = 86
RETURN count(*) AS size,
       sum(CASE WHEN u.is_bot = true THEN 1 ELSE 0 END) * 1.0 / count(*) AS botRatio

```

B. Stance Distribution

```

MATCH (u:User)
WHERE u.cluster = 86
RETURN u.stance AS stance, count(*) AS count
ORDER BY count DESC

```

17. Reciprocity: How Many FOLLOWS Go Back from Targeted Community

```

MATCH (u:User)-[:FOLLOWS]->(v:User)
WHERE u.cluster = 86 AND v.cluster = 39
RETURN count(*) AS edges86_to_39

```

18. Which Target Clusters Reciprocate Bot-Follows Most?

```

MATCH (b:User)-[:FOLLOWS]->(h:User)
WHERE b.cluster = 39 AND h.cluster <> 39
WITH h.cluster AS targetCluster, count(*) AS outCount
MATCH (h2:User)-[:FOLLOWS]->(b2:User)
WHERE h2.cluster = targetCluster AND b2.cluster = 39
WITH targetCluster, outCount, count(*) AS inCount
RETURN targetCluster, outCount, inCount,
       toFloat(inCount)/outCount AS reciprocity
ORDER BY reciprocity DESC
LIMIT 5

```

19. Community Cohesion

Shows what proportion of a community's edges stay inside the community.

```

MATCH (u:User)-[:FOLLOWS]->(v:User)
WITH u.cluster AS cluster,
     sum(CASE WHEN u.cluster = v.cluster THEN 1 ELSE 0 END) AS intraEdges,
     count(*) AS totalEdges
RETURN cluster, toFloat(intraEdges) / totalEdges AS cohesionRatio, totalEdges
ORDER BY cohesionRatio DESC
LIMIT 10

```

20. BridgeNodes (High InterCluster Degree)

Finds users who follow the most outside their own cluster.

```
MATCH (u:User)-[:FOLLOWS]->(v:User)
```

```
WITH u.user_id AS user, u.cluster AS cluster,
```

```
    sum(CASE WHEN u.cluster <> v.cluster THEN 1 ELSE 0 END) AS interDeg
```

```
RETURN user, cluster, interDeg
```

```
ORDER BY interDeg DESC
```

```
LIMIT 10
```